Aircraft Performance Engine

Takeoff & Landing Performance Model

Engineering Method, Implementation and Worked Examples

Atmosphere, aircraft, takeoff, Balanced Field Length, landing and runway analysis,
organised by Python file and function, with worked examples for the
Bombardier Challenger 3500 and the Cirrus SR22T

Mark Barsoum

November 2026

Summary

This project estimates takeoff and landing distances for three aircraft: the Cessna 172S, Cirrus SR22T and Bombardier Challenger 3500. The calculation combines an atmosphere model, aircraft force models and numerical integration of the ground roll. Its purpose is to study how aircraft characteristics and runway conditions affect the distance required.

The method is documented alongside the Python functions that implement it. Shared equations are evaluated in one place, while aircraft subclasses supply the appropriate thrust model and operating assumptions. This keeps the takeoff, rejected-takeoff and landing calculations consistent.

The worked examples compare the Challenger 3500 and SR22T at KICT under the same weather conditions. They show the inputs, intermediate calculations and runway margins. Numerical tables are supplied by `report/make_examples.py`; they must be regenerated whenever model inputs or equations change. The report describes a simplified engineering model, not approved aircraft performance data.

Contents

Summary . . . . . **i**

1 Introduction **1**
- 1.1 Purpose 1
- 1.2 How the Code Is Organised 1
- 1.3 Conventions 1

2 Computation Procedure **2**

3 Nomenclature **2**

I Methodology by Python File **4**

4 `units.py`: Unit Conversion **5**

5 `atmosphere.py`: International Standard Atmosphere **5**
- 5.1 `isa()` 5
- 5.2 `sigma()` 6
- 5.3 `speed_of_sound()` 6

6 `aircraft.py`: Aircraft Model **6**
- 6.1 `Aircraft.lift()` and `Aircraft.drag()` 6
- 6.2 `Aircraft.stall_speed()` 7
- 6.3 `Aircraft.engine_fraction()` 7
- 6.4 `PropAircraft.thrust()` 7
- 6.5 `JetAircraft.thrust()` 8
- 6.6 `cl_braking()` and `reverse_thrust()` 8

7 `segments.py`: Conditions and Equations of Motion **9**
- 7.1 `make_conditions()` 9
- 7.2 `accelerate()` 9
- 7.3 `decelerate()` 10
- 7.4 Numerical Integration 10
- 7.5 `climb_to_screen()` 10

8 `takeoff.py`: All-Engines Takeoff Distance **11**
- 8.1 `takeoff_speeds()` 11
- 8.2 `rotation_distance()` 11
- 8.3 `takeoff()` 12

9 `bfl.py`: Balanced Field Length and Takeoff Field Length **12**
- 9.1 `accelerate_go()` 12
- 9.2 `accelerate_stop()` 13
- 9.3 `balanced_field()` 13
- 9.4 `plot_bfl()` 13

10 `landing.py`: Landing Distance **14**
- 10.1 `landing()` 14

11 `airports.py`: Runways and Wind Components . . . **14**
11.1 `Runway` and `Airport` . . . 14
11.2 `runway_pair()` . . . 15
11.3 `wind_components()` . . . 15
12 `runway_analysis.py`: Runway Checks . . . **15**
12.1 `isa_dev_from_oat()` . . . 15
12.2 `factored_headwind()` . . . 16
12.3 `analyze_runway()` . . . 16
13 `sensitivity.py`: Parameter Sweeps . . . **16**
13.1 `takeoff_metric_m()` and `landing_metric_m()` . . . 16
13.2 `run_sweeps()` . . . 17
14 Supporting Files . . . **17**

II Worked Examples **18**

15 Example A: Bombardier Challenger 3500 . . . **19**
15.1 Inputs . . . 19
15.2 Step 1: Atmosphere . . . 19
15.3 Step 2: Wind . . . 20
15.4 Step 3: Weight and Takeoff Speeds . . . 20
15.5 Step 4: Forces on the Ground Roll . . . 20
15.6 Step 5: Ground Roll by Numerical Integration . . . 21
15.7 Step 6: Air Distance to the Screen Height . . . 22
15.8 Step 7: Balanced Field Length . . . 22
15.9 Step 8: Landing . . . 24
15.10 Step 9: Runway Check . . . 24
16 Example B: Cirrus SR22T . . . **25**
16.1 Inputs . . . 25
16.2 Step 1: Atmosphere . . . 25
16.3 Step 2: Wind . . . 26
16.4 Step 3: Weight and Takeoff Speeds . . . 26
16.5 Step 4: Forces on the Ground Roll . . . 26
16.6 Step 5: Ground Roll by Numerical Integration . . . 27
16.7 Step 6: Air Distance to the Screen Height . . . 28
16.8 Step 7: Engine-Failure Case . . . 28
16.9 Step 8: Landing . . . 29
16.10 Step 9: Runway Check . . . 29
17 Assumptions and Limitations . . . **29**
References . . . **30**
A Model Constants . . . **31**

List of Figures

1 Challenger 3500: Balanced Field Length (BFL) for this example. 23

List of Tables

1 Python files and their role. 1
2 Nomenclature. 3
3 Conversion functions in `units.py`. 5
4 Runway checks by aircraft type. 16
5 Challenger 3500: example inputs. 19
6 Challenger 3500: forces on the ground roll. 21
7 Challenger 3500: ground-roll time history (RK45 output). 21
8 Challenger 3500: accelerate-go and accelerate-stop against trial V_1. 22
9 Challenger 3500: runway check, KICT runway 01L. 24
10 Cirrus SR22T: example inputs. 25
11 Cirrus SR22T: forces on the ground roll. 27
12 Cirrus SR22T: ground-roll time history (RK45 output). 27
13 Cirrus SR22T: runway check, KICT runway 01L. 29
14 Constants used in the model. 31

1 Introduction

1.1 Purpose

The main question is how much runway an aircraft needs at a specified weight, temperature, elevation and wind. The all-engines takeoff calculation answers only part of that question for a twin. An engine failure introduces a choice between continuing and stopping, so both distances must be considered. The model applies selected factors from 14 CFR Part 25 and CAR 525, subject to the assumptions listed at the end of this report.

1.2 How the Code Is Organised

The program separates aircraft data, flight-segment calculations and user interaction. Table 1 provides a map from each engineering topic to its implementation. The sections that follow use the same module and function names as the source code.

Section	File	Role
4	`units.py`	Unit conversion functions
5	`atmosphere.py`	International Standard Atmosphere (ISA)
6	`aircraft.py`	Aircraft classes: aerodynamics and thrust
7	`segments.py`	Day conditions and the equations of motion
8	`takeoff.py`	All-engines takeoff distance
9	`bfl.py`	Balanced Field Length and Takeoff Field Length
10	`landing.py`	Landing distance and Landing Distance Required
11	`airports.py`	Runways, declared distances and wind components
12	`runway_analysis.py`	Runway checks with regulatory wind factors
13	`sensitivity.py`	Parameter sweeps
14	others	Exceptions, profiles, charts and the menu

Table 1. Python files and their role.

1.3 Conventions

- All calculations are in SI units. Feet, knots, pounds and degrees Celsius are used only for input and output.
- **All speeds are true airspeed (TAS)**, labelled KTAS when shown in knots.
- Sources are cited by number in IEEE style, e.g. [1]; full citations are in the References, numbered in order of first citation.

2 Computation Procedure

A runway assessment combines the following calculations. The list describes the engineering dependencies; the program may repeat a segment when searching for balanced V_1.

1. **Day conditions.** `isa_dev_from_oat()` (12.1); `isa()` (5.1)–(5.4); `wind_components()` and `factored_headwind()` (11.3)–(12.2); `make_conditions()` (7.1)–(7.3).
2. **Speeds.** `stall_speed()` (6.4); `takeoff_speeds()` (8.1).
3. **All-engines takeoff.** `accelerate()` (7.4) to V_R; `rotation_distance()` (8.2); `climb_to_screen()` (7.8)–(7.14); TOD (8.4).
4. **Engine failure (multi-engine only).** `accelerate_go()` (9.1) and `accelerate_stop()` (9.2) for trial V_1; Brent's method on (9.3); BFL (9.4); TOFL (9.5).
5. **Landing.** `landing()`: (10.1)–(10.4), with `decelerate()` (7.5); LDR (10.5).
6. **Runway check.** Compare against TORA, TODA, ASDA and LDA (Table 4).

3 Nomenclature

Symbol	Meaning	Unit
a	Speed of sound	m/s
C_L, C_D	Lift and drag coefficients	–
C_{D_0}	Zero-lift (parasite) drag coefficient	–
$C_{L,\mathrm{gnd}}$	Lift coefficient during the ground roll	–
$C_{L,\max}$	Maximum lift coefficient (takeoff or landing flaps)	–
D, L, T, W	Drag, lift, thrust, weight	N
g_0	Standard gravity, 9.80665	m/s^2
h	Pressure altitude / field elevation	m
h_{scr}	Screen height	m
h_{crit}	Turbocharger critical altitude	m
k	Induced-drag factor	–
k_{lapse}	Turbofan thrust lapse with Mach number	–
L_{r}	Temperature lapse rate, 0.0065	K/m
M	Mach number	–
m	Aircraft mass	kg
n	Load factor in the transition arc	–

Nomenclature (continued)

Symbol	Meaning	Unit
N_e, N_{op}	Number of engines; engines operating	–
P	Shaft power	W
p	Static pressure	Pa
q	Dynamic pressure, $\frac{1}{2}\rho V^2$	Pa
R	Specific gas constant of air, 287.05	J/(kg K)
R_{TR}	Transition arc radius	m
S	Reference wing area	m^2
s	Distance along the runway	m
T_{SLS}	Sea-level static thrust, all engines	N
Θ	Static air temperature	K
ΔT	ISA deviation	°C
ΔT_{flat}	Flat-rating temperature	°C
V, V_{TAS}	True airspeed	m/s
V_{GS}	Ground speed	m/s
V_w, V_x	Headwind (negative = tailwind) and crosswind components	m/s
V_{S1}, V_{S0}	Stall speed, takeoff and landing configuration	m/s
V_1, V_R, V_2	Decision, rotation and takeoff safety speeds	m/s
V_{REF}, V_{TD}	Landing reference and touchdown speeds	m/s
γ	Climb angle	rad
γ_{air}	Ratio of specific heats, 1.4	–
η_p	Propeller efficiency	–
θ	Runway slope angle (+ uphill)	rad
μ_{roll}, μ_{brake}	Rolling and braking friction coefficients	–
ρ	Air density	$\mathrm{kg/m}^3$
σ	Density ratio, ρ/ρ_{SL}	–

Table 2. Nomenclature.

PART I

Methodology by Python File

4 `units.py`: Unit Conversion

Every conversion in the project goes through this file, so each factor is defined exactly once [2]. Each function accepts a single number or a NumPy array.

Functions	Conversion	Factor
`ft_to_m()` / `m_to_ft()`	feet ↔ metres	1 ft = 0.3048 m (exact)
`ft2_to_m2()` / `m2_to_ft2()`	ft^2 ↔ m^2	0.3048^2
`kt_to_mps()` / `mps_to_kt()`	knots ↔ m/s	1 kt = 1852/3600 m/s (exact)
`lb_to_kg()` / `kg_to_lb()`	pound-mass ↔ kg	1 lb = 0.45359237 kg (exact)
`lbf_to_n()` / `n_to_lbf()`	pound-force ↔ N	1 lbf = 4.4482216 N
`hp_to_w()` / `w_to_hp()`	horsepower ↔ W	1 hp = 745.69987 W
`c_to_k()` / `k_to_c()`	°C ↔ K	+273.15

Table 3. Conversion functions in `units.py`.

5 `atmosphere.py`: International Standard Atmosphere

The atmosphere calculation uses the tropospheric relations in [3, Ch. 3] and the constants in [4]. Its altitude range is $0 \le h \le 11\,000$ m. The sea-level reference values are $\Theta_{SL} = 288.15$ K, $p_{SL} = 101\,325$ Pa and $\rho_{SL} = 1.225$ kg/m^3.

5.1 `isa()`

`isa(alt_m, isa_dev_c=0) → (temp_k, press_pa, rho_kg_m3)`
Returns: temperature, pressure and density at a pressure altitude.

Pressure altitude first determines the standard temperature and pressure. The ISA deviation is then added to temperature, and density is calculated using the ideal-gas relation. This order matters: changing the day's temperature does not change the pressure assigned to a given pressure altitude.

$$\Theta_{std} = \Theta_{SL} - L_r\, h \tag{5.1}$$

$$p = p_{SL}\left(\frac{\Theta_{std}}{\Theta_{SL}}\right)^{g_0/(L_r R)}, \qquad \frac{g_0}{L_r R} \approx 5.2559 \tag{5.2}$$

$$\Theta = \Theta_{std} + \Delta T \tag{5.3}$$

$$\rho = \frac{p}{R\,\Theta} \tag{5.4}$$

The function rejects altitudes outside this range with `AtmosphereRangeError`.

5.2 `sigma()`

`sigma(alt_m, isa_dev_c=0)` → `float`
Returns: the density ratio σ.

$$\sigma = \frac{\rho}{\rho_{SL}} \tag{5.5}$$

5.3 `speed_of_sound()`

`speed_of_sound(alt_m, isa_dev_c=0)` → `float`
Returns: the speed of sound [m/s].

$$a = \sqrt{\gamma_{air} R \Theta} \tag{5.6}$$

6 `aircraft.py`: Aircraft Model

The `Aircraft` class stores mass limits, wing area, aerodynamic coefficients and friction coefficients. It also provides the lift, drag and stall-speed methods used by both aircraft types. Inheritance is used only where behaviour differs:

- `PropAircraft`: propeller thrust (normally aspirated or turbocharged), 50 ft takeoff screen height [5], 1 s rotation, 2 s free roll.
- `JetAircraft`: flat-rated turbofan thrust, ground spoilers and reverse thrust, 35 ft takeoff screen height [6], [7], 3 s rotation, 3 s free roll, LDR rule.

6.1 `Aircraft.lift()` and `Aircraft.drag()`

`lift(tas_mps, rho_kg_m3, cl), drag(tas_mps, rho_kg_m3, cl)` → `float`
Returns: lift and drag [N].

Dynamic pressure is evaluated at TAS. Lift follows from the selected C_L, and drag is obtained from

the parabolic polar in [8, Ch. 6]:

$$q = \tfrac{1}{2}\rho V^2 \tag{6.1}$$

$$L = q\,S\,C_L \tag{6.2}$$

$$D = q\,S\left(C_{D_0} + k\,C_L^2\right) \tag{6.3}$$

6.2 `Aircraft.stall_speed()`

`stall_speed(weight_n, rho_kg_m3, cl_max) → float`
Returns: stall speed, TAS [m/s].

$$V_S = \sqrt{\frac{2W}{\rho\,S\,C_{L,\max}}} \tag{6.4}$$

V_{S1} uses $C_{L,\max}$ with takeoff flaps; V_{S0} uses landing flaps.

6.3 `Aircraft.engine_fraction()`

`engine_fraction(engines_operating) → float`
Returns: the share of total thrust available.

$$T = T_{\text{AEO}}\,\frac{N_{\text{op}}}{N_e}, \qquad N_{\text{op}} = N_e - 1 \text{ with one engine inoperative (OEI)} \tag{6.5}$$

6.4 `PropAircraft.thrust()`

`thrust(tas_mps, alt_m, isa_dev_c, engines_operating) → float`
Returns: propeller thrust [N].

The propeller model converts available shaft power to thrust using a constant efficiency. The relation $T = \eta_p P/V$ is unsuitable at zero airspeed, so the implementation limits thrust to the static value and limits the denominator to 1 m/s [9, 8]. Available power follows density for a normally aspirated engine. For a turbocharged engine, rated power is retained up to the critical altitude and reduced above it:

$$\sigma_P = \begin{cases} \sigma(h, \Delta T), & \text{normally aspirated} \\ 1, & \text{turbocharged, } h \le h_{\text{crit}} \\ \sigma(h, \Delta T)/\sigma(h_{\text{crit}}, \Delta T), & \text{turbocharged, } h > h_{\text{crit}} \end{cases} \tag{6.6}$$

$$T_{\text{prop}} = \min\left(\sigma_P\, T_{\text{SLS}},\ \frac{\eta_p\, \sigma_P\, P_{\max}}{\max(V,\ 1)}\right) \frac{N_{\text{op}}}{N_e} \tag{6.7}$$

6.5 `JetAircraft.thrust()`

`thrust(tas_mps, alt_m, isa_dev_c, engines_operating)` → `float`
Returns: turbofan thrust [N].

The turbofan approximation separates the flat-rating temperature allowance from the altitude correction. The effective density ratio uses only the ISA deviation above the flat-rating limit. The Mach correction still uses the actual local speed of sound. The resulting thrust relation is a simplified representation based on [10]:

$$\Delta T_{\text{excess}} = \max\left(\Delta T - \Delta T_{\text{flat}},\ 0\right) \tag{6.8}$$

$$\sigma_{\text{eff}} = \sigma(h,\ \Delta T_{\text{excess}}) \tag{6.9}$$

$$T_{\text{jet}} = T_{\text{SLS}}\, \sigma_{\text{eff}}^{0.7} \left(1 - k_{\text{lapse}}\, \frac{V}{a(h, \Delta T)}\right) \frac{N_{\text{op}}}{N_e} \tag{6.10}$$

6.6 `cl_braking()` and `reverse_thrust()`

`cl_braking()` → `float`; `reverse_thrust(tas_mps, use_reverse)` → `float`
Returns: lift coefficient while braking; reverse thrust [N].

Braking lift depends on the aircraft subclass. `JetAircraft` returns zero to represent deployed ground spoilers; `PropAircraft` retains $C_{L,\text{gnd}}$. When reverse thrust is enabled for the jet, the model uses a fixed fraction of SLS thrust above the cutoff speed [11]:

$$T_{\text{rev}} = \begin{cases} r_{\text{rev}}\, T_{\text{SLS}}, & V > 60\,\text{kt} \\ 0, & \text{otherwise} \end{cases} \tag{6.11}$$

The default runway analysis and the accelerate-stop calculation use no reverse-thrust credit. A separate landing calculation can enable reverse thrust to examine its effect. The dry landing-distance treatment is based on [6, §25.125].

7 `segments.py`: Conditions and Equations of Motion

This module contains the shared acceleration, braking and climb calculations. A `Conditions` object passes density, weight, wind, slope and friction to each segment. Keeping these values together avoids separate condition calculations in `takeoff.py`, `bfl.py` and `landing.py`.

7.1 `make_conditions()`

`make_conditions(acft, mass_kg, elev_m, isa_dev_c, hw_kt, slope_pct, rwy_condition)` → `Conditions`
Returns: the day's density, weight, wind, slope and braking friction.

The input mass is converted to weight using $W = m\,g_0$, and `isa()` supplies density. The runway gradient is converted from percent to an angle. A wet runway reduces the braking coefficient as follows:

$$\theta = \arctan\left(\frac{\text{slope}_{\%}}{100}\right) \tag{7.1}$$

$$\mu_{\text{brake,wet}} = 0.5\,\mu_{\text{brake}} \tag{7.2}$$

the slope term following [12] and the wet-runway factor a simplification of [11]. The state is tracked in ground speed, while every aerodynamic and thrust force uses true airspeed:

$$V_{\text{TAS}} = V_{\text{GS}} + V_w \tag{7.3}$$

7.2 `accelerate()`

`accelerate(acft, cond, tas_target_mps, gs_start_mps, dist_start_m, engines_operating)` → `(dist_end_m, time_s, solution)`
Returns: distance and time when the target airspeed is reached.

The accelerating ground roll [8, Ch. 6], with $C_L = C_{L,\text{gnd}}$ and the slope term [12]:

$$m\,\frac{dV_{\text{GS}}}{dt} = T(V_{\text{TAS}}) - D(V_{\text{TAS}}) - \mu_{\text{roll}}\,(W - L(V_{\text{TAS}})) - W\sin\theta, \qquad \frac{ds}{dt} = V_{\text{GS}} \tag{7.4}$$

At rest, a negative calculated acceleration is replaced by zero to prevent backward motion. The failure condition is reported as `InsufficientThrustError` if the target TAS is not reached within the integration time limit. In the implementation, the wheel load is limited to $\max(W - L, 0)$ before rolling or braking friction is calculated.

7.3 `decelerate()`

`decelerate(acft, cond, gs_start_mps, dist_start_m, cl, use_reverse)` → `(dist_end_m, time_s, solution)`
Returns: distance and time to a stop.

The braking roll at idle thrust (taken as zero) [8, Ch. 6]; [1, Ch. 17]:

$$m\,\frac{dV_{\mathrm{GS}}}{dt} = -\Big[D + \mu_{\mathrm{brake}}\,(W - L) + T_{\mathrm{rev}} + W\sin\theta\Big] \tag{7.5}$$

7.4 Numerical Integration

The solver advances distance and ground speed together as $\mathbf{y} = [s, V_{\mathrm{GS}}]^{\mathsf{T}}$. Both ground segments use SciPy's adaptive RK45 method through `solve_ivp`. The maximum time step is 0.5 s, and the integration interval is limited to 300 s. Event functions determine the endpoint instead of selecting the nearest stored time sample:

$$g_{\mathrm{accel}}(t, \mathbf{y}) = (V_{\mathrm{GS}} + V_w) - V_{\mathrm{target}} \tag{7.6}$$

$$g_{\mathrm{stop}}(t, \mathbf{y}) = V_{\mathrm{GS}} - 0.1 \tag{7.7}$$

The stopping event uses 0.1 m/s as the practical standstill threshold. Acceleration detects an upward crossing of the target TAS; braking detects a downward crossing of the stopping threshold.

7.5 `climb_to_screen()`

`climb_to_screen(acft, cond, vs1_mps, screen_height_ft, engines_operating)` → `(air_dist_m, climb_grad_pct)`
Returns: air distance from liftoff to the screen height, and the climb gradient.

The airborne segment follows Raymer's transition-arc approximation [1, Ch. 17]. The calculation first evaluates thrust and drag at $V_2 = 1.2V_{S1}$, using the lift coefficient required to support weight. Excess thrust determines the climb angle:

$$C_{L,\mathrm{climb}} = \frac{W}{q_{V_2}\,S} \tag{7.8}$$

$$\sin\gamma = \frac{T(V_2) - D(V_2,\,C_{L,\mathrm{climb}})}{W}, \qquad \text{gradient} = 100\tan\gamma\ [\%] \tag{7.9}$$

A non-positive excess-thrust ratio raises `ClimbPerformanceError`. For a positive ratio, the code bounds the inverse-sine input at 0.99. The transition is then represented by a circular arc at

$V_{\mathrm{TR}} = 1.15 V_{S1}$ and load factor $n = 1.2$:

$$R_{\mathrm{TR}} = \frac{V_{\mathrm{TR}}^2}{g_0\,(n-1)} \tag{7.10}$$

$$h_{\mathrm{TR}} = R_{\mathrm{TR}}\,(1 - \cos\gamma) \tag{7.11}$$

If the screen height is reached inside the arc ($h_{\mathrm{TR}} \geq h_{\mathrm{scr}}$):

$$s_{\mathrm{air}} = \sqrt{R_{\mathrm{TR}}^2 - (R_{\mathrm{TR}} - h_{\mathrm{scr}})^2} \tag{7.12}$$

otherwise the arc is followed by a steady climb:

$$s_{\mathrm{air}} = R_{\mathrm{TR}} \sin\gamma + \frac{h_{\mathrm{scr}} - h_{\mathrm{TR}}}{\tan\gamma} \tag{7.13}$$

The final correction converts the distance through the air mass to distance over the ground using the transition speed:

$$s_{\mathrm{air}} \leftarrow s_{\mathrm{air}}\,\frac{V_{\mathrm{TR}} - V_w}{V_{\mathrm{TR}}} \tag{7.14}$$

8 `takeoff.py`: All-Engines Takeoff Distance

8.1 `takeoff_speeds()`

`takeoff_speeds(acft, cond)` → `(vs1_mps, vr_mps, v2_mps)`
Returns: the takeoff V-speeds (TAS).

$$V_R = 1.1\,V_{S1}, \qquad V_2 = 1.2\,V_{S1}, \qquad V_{\mathrm{TR}} = 1.15\,V_{S1} \tag{8.1}$$

Ratios from [1, Ch. 17], consistent with [6, §25.107.]

8.2 `rotation_distance()`

`rotation_distance(acft, cond, vr_mps)` → `float`
Returns: ground distance while rotating from V_R to liftoff [m].

The aircraft rotates for t_{rot} (1 s light aircraft, 3 s jets [1]) at roughly constant speed:

$$s_{\mathrm{rot}} = (V_R - V_w)\,t_{\mathrm{rot}} \tag{8.2}$$

8.3 `takeoff()`

`takeoff(acft, elev_m, isa_dev_c, mass_kg, hw_kt, slope_pct, rwy_condition)` → `dict`
Returns: V-speeds, ground roll, air distance, climb gradient and TOD.

1. `make_conditions()` builds the day's conditions.
2. `takeoff_speeds()` gives V_{S1}, V_R and V_2.
3. `accelerate()` integrates (7.4) from rest to V_R; `rotation_distance()` adds the rotation:

$$s_{\text{ground}} = s(V_R) + s_{\text{rot}} \tag{8.3}$$

4. `climb_to_screen()` gives the air distance to 50 ft (props) or 35 ft (jets).

$$\text{TOD}_{\text{AEO}} = s_{\text{ground}} + s_{\text{air}} \tag{8.4}$$

9 `bfl.py`: Balanced Field Length and Takeoff Field Length

The BFL calculation follows the continue-or-stop approach described in [13, 14, 10, 11], with the selected distance rules from [6, §§25.107–25.113]. For each trial V_1, the model assumes that one engine fails at that speed. Thus $V_{\text{EF}} = V_1$ in this implementation. The following functions evaluate the two outcomes.

9.1 `accelerate_go()`

`accelerate_go(acft, cond, v1_mps)` → `(distance_m, oei_climb_grad_pct)`
Returns: distance to 35 ft when the takeoff continues after the failure.

$$s_{\text{go}}(V_1) = \underbrace{s_{\text{AEO}}(0 \to V_1)}_{\text{all engines}} + \underbrace{s_{\text{OEI}}(V_1 \to V_R)}_{\text{one engine out}} + s_{\text{rot}} + s_{\text{air,OEI}} \tag{9.1}$$

The first ground segment uses all engines. The second segment and the climb use one fewer operating engine through (6.5). Rotation is added at the scheduled V_R.

9.2 accelerate_stop()

`accelerate_stop(acft, cond, v1_mps)` → `float`
Returns: distance to a full stop when the takeoff is rejected.

The rejected takeoff includes a 2-second distance allowance at the decision-speed ground speed, following the model's treatment of §25.109. Braking then uses the aircraft's braking lift coefficient, zero idle thrust and no reverse-thrust credit:

$$s_{\text{stop}}(V_1) = s_{\text{AEO}}(0 \rightarrow V_1) + (V_1 - V_w)\,(2\,\text{s}) + s_{\text{brake}}(V_1 \rightarrow 0) \tag{9.2}$$

9.3 balanced_field()

`balanced_field(acft, elev_m, isa_dev_c, mass_kg, hw_kt, slope_pct, rwy_condition)` → `dict`
Returns: balanced V_1, BFL, AEO TOD, TOFL and the curves for the chart.

An earlier failure increases the distance spent accelerating with one engine inoperative. A later rejection increases the distance needed to stop. The solver searches for the decision speed at which these distances are equal:

$$f(V_1) = s_{\text{go}}(V_1) - s_{\text{stop}}(V_1) = 0, \qquad 0.5\,V_R \le V_1 \le V_R \tag{9.3}$$

Brent's method (`scipy.optimize.brentq`) solves this equation with a tolerance of 0.01 m/s. The implementation raises the lower bound to $V_w + 1$ m/s if that exceeds $0.5V_R$. The upper bound is V_R, consistent with the model's use of §25.107. When no crossing lies inside the interval, the appropriate endpoint is used and the result is marked unbalanced. VMCG is not calculated. At a balanced solution,

$$\text{BFL} = s_{\text{go}}(V_1) = s_{\text{stop}}(V_1) \tag{9.4}$$

§25.113 requires the takeoff distance to be the greater of the engine-out distance and 115% of the all-engines distance:

$$\text{TOFL} = \max\left(\text{BFL},\ 1.15 \times \text{TOD}_{\text{AEO}}\right) \tag{9.5}$$

9.4 plot_bfl()

`plot_bfl(result, filename, runway, extra_conditions)` → `filename`
Returns: the balanced-field chart.

The chart presents both distance curves against V_1, with markers for the balanced point and V_R. The factored AEO distance is drawn on the same axes. If a runway is supplied, its TODA and ASDA are

included as reference lines. The plotting code retains the shared project colours, fonts and sidebar layout.

10 `landing.py`: Landing Distance

10.1 `landing()`

`landing(acft, elev_m, isa_dev_c, mass_kg, hw_kt, use_reverse, slope_pct, rwy_condition)` → `dict`

Returns: landing speeds, air distance, free roll, braking roll, LD and LDR.

Method: [8, Ch. 6]; [1, Ch. 17]; [6, §25.125]; [15, §121.195.] The landing screen height is 50 ft for every aircraft.

$$V_{REF} = 1.3\,V_{S0}, \qquad V_{TD} = 1.15\,V_{S0} \tag{10.1}$$

$$s_{air} = \frac{h_{scr}}{\tan 3^\circ} \cdot \frac{V_{REF} - V_w}{V_{REF}} \tag{10.2}$$

$$s_{free} = (V_{TD} - V_w)\,t_{free}, \qquad t_{free} = 2\,\text{s (prop)},\ 3\,\text{s (jet)} \tag{10.3}$$

$$LD = s_{air} + s_{free} + s_{brake} \tag{10.4}$$

where s_{brake} is (7.5) integrated by `decelerate()` from $V_{TD} - V_w$ to rest. For jets, §121.195 requires the landing to fit within 60% of the runway, with a further 15% on a wet runway:

$$LDR_{dry} = \frac{LD_{dry}}{0.6}, \qquad LDR_{wet} = 1.15 \times LDR_{dry} \tag{10.5}$$

11 `airports.py`: Runways and Wind Components

11.1 `Runway` and `Airport`

A `Runway` object represents one direction of operation. The opposite direction can have a different slope and displaced threshold, so it is stored separately. The available distances are TORA, TODA, ASDA and LDA, using the terminology in [16].

11.2 `runway_pair()`

`runway_pair(id_1, hdg_1, id_2, hdg_2, length_ft, disp_1_ft, disp_2_ft, slope_1_pct) → [Runway, Runway]`
Returns: both directions of one physical runway.

The built-in runway data are simplified from runway length ℓ, displaced threshold d and threshold elevations [17]. These are model inputs rather than verified operational declared distances:

$$\text{TORA} = \text{TODA} = \text{ASDA} = \ell, \qquad \text{LDA} = \ell - d \tag{11.1}$$

$$\text{slope}_{\%} = 100\,\frac{h_{\text{far end}} - h_{\text{near end}}}{\ell} \tag{11.2}$$

11.3 `wind_components()`

`wind_components(wind_dir_deg, wind_kt, rwy_hdg_deg) → (hw_kt, xw_kt)`
Returns: headwind and crosswind components.

For a wind from ψ_w on a runway of true heading ψ_r [11]:

$$V_w = V_{\text{wind}}\cos(\psi_w - \psi_r) \qquad (+\ \text{headwind}) \tag{11.3}$$

$$V_x = V_{\text{wind}}\sin(\psi_w - \psi_r) \qquad (+\ \text{from the right}) \tag{11.4}$$

12 `runway_analysis.py`: Runway Checks

12.1 `isa_dev_from_oat()`

`isa_dev_from_oat(elev_m, oat_c) → float`
Returns: ISA deviation [°C].

$$\Delta T = \text{OAT} - (\Theta_{\text{std}}(h) - 273.15) \tag{12.1}$$

12.2 `factored_headwind()`

`factored_headwind(hw_kt)` → `float`
Returns: the headwind used for performance.

No more than 50% of a headwind may be credited and at least 150% of a tailwind must be applied [6, §25.105(d)](1):

$$V_{w,\text{perf}} = \begin{cases} 0.5\,V_w, & V_w \geq 0 \\ 1.5\,V_w, & V_w < 0 \end{cases} \tag{12.2}$$

12.3 `analyze_runway()`

`analyze_runway(acft, apt, rwy, oat_c, wind_dir_deg, wind_kt, rwy_condition, tow_kg, lw_kg)` → `dict`
Returns: every distance and a pass or fail for takeoff and landing.

The analysis first converts OAT to ISA deviation and resolves the reported wind along the runway. It then applies the performance wind factor and runway slope to both takeoff and landing. The resulting requirements are compared with the available distances in Table 4. A positive margin means that the modelled requirement fits within the stated available distance.

Aircraft	Takeoff Checks	Landing Check
Multi-engine	TOFL ≤ TODA and ASD ≤ ASDA	LDR ≤ LDA (jets)
Single-engine	TOD ≤ TODA and $s_{\text{ground}} \leq$ TORA	LD ≤ LDA

Table 4. Runway checks by aircraft type.

13 `sensitivity.py`: Parameter Sweeps

13.1 `takeoff_metric_m()` and `landing_metric_m()`

`takeoff_metric_m(acft, **conditions)`, `landing_metric_m(acft, **conditions)` → `float`
Returns: the distance that matters for the aircraft [m].

The sweep uses TOFL for a multi-engine aircraft and AEO TOD for a single-engine aircraft. Landing is compared using unfactored LD with reverse thrust disabled. Keeping these measures fixed makes the effect of each input easier to interpret.

13.2 `run_sweeps()`

`run_sweeps(acft)` → `dict`
Returns: the distances over each sweep.

Each sweep starts from sea level, ISA temperature, MTOW for takeoff, MLW for landing, calm wind and a dry, level runway. Only one input changes at a time. The ranges are 0–8 000 ft elevation, −20 to +30 °C ISA deviation, 70–100% weight and −10 to +20 kt headwind. Percentage changes use the baseline distance as the denominator:

$$\Delta_{\%} = 100 \left(\frac{s}{s_{\text{baseline}}} - 1 \right) \tag{13.1}$$

14 Supporting Files

The remaining modules handle input, storage, presentation and error reporting. They do not introduce additional performance equations.

- `exceptions.py`: `PerformanceError` and its subclasses `InsufficientThrustError`, `ClimbPerformanceError` and `AtmosphereRangeError`, so the menu can report exactly why a case cannot be computed.
- `profiles.py`: create, save and load aircraft profiles (JSON), rebuilt as `PropAircraft` or `JetAircraft`.
- `plot_style.py`: the shared chart style.
- `main.py`: the menu that ties everything together.

PART II

Worked Examples

The two worked examples use a common runway and weather case. This allows the effect of the aircraft model to be followed through the same calculation stages:

- Wichita Dwight D. Eisenhower National (KICT), runway 01L, elevation 1 333 ft
- Outside air temperature 30 °C, wind 20° at 10 kt (straight down the runway)
- Dry runway, slope taken from the threshold elevations
- Takeoff at MTOW, landing at MLW

The numerical substitutions and tables are generated by `report/make_examples.py`. Displayed values are rounded; the Python calculations retain full numerical precision.

15 Example A: Bombardier Challenger 3500

15.1 Inputs

Table 5 summarises the aircraft inputs and operating conditions. The calculation is for Wichita Dwight D. Eisenhower National (KICT) runway 01L.

Input	Value
Maximum takeoff weight (MTOW)	40 600 lb = 18 415.9 kg
Maximum landing weight (MLW)	34 150 lb = 15 490.2 kg
Reference wing area, S	48.50 m^2 (522.0 ft^2)
C_{D_0} / k	0.030 / 0.045
$C_{L,\max}$ takeoff / landing	1.57 / 2.30
$C_{L,\text{gnd}}$	0.30
μ_{roll} / μ_{brake}	0.02 / 0.40
Sea-level static thrust, T_{SLS}	65 149 N (14 646 lbf)
Number of engines, N_e	2
Thrust lapse, k_{lapse}	0.90
Flat rating, ΔT_{flat}	ISA 15 °C
Reverse thrust ratio	0.30
Field elevation, h	1 333 ft = 406.3 m
Outside air temperature (OAT)	30 °C
Reported wind	20° / 10 kt
Runway true heading	20°
Runway slope (direction of travel)	0.165 %
Runway condition	Dry
TODA = ASDA / LDA	10 302 ft / 10 302 ft

Table 5. Challenger 3500: example inputs.

15.2 Step 1: Atmosphere

Standard temperature at the field elevation, then the ISA deviation from the OAT ((5.1) and (12.1)):

$$\Theta_{\text{std}} = 288.15 - 0.0065 \times 406.3 = 285.51\ \text{K}$$
$$\Delta T = 30.0 - (285.51 - 273.15) = 17.64\ ^\circ\text{C} \quad (\text{ISA } + 17.6)$$

Pressure, temperature, density, density ratio and speed of sound ((5.2)–(5.6)):

$$p = 101\,325\left(\frac{285.51}{288.15}\right)^{5.2559} = 96\,538\ \text{Pa}$$
$$\Theta = 285.51 + 17.64 = 303.15\ \text{K}$$
$$\rho = \frac{96\,538}{287.05 \times 303.15} = 1.1094\ \text{kg/m}^3$$
$$\sigma = \frac{1.1094}{1.225} = 0.9056$$
$$a = \sqrt{1.4 \times 287.05 \times 303.15} = 349.0\ \text{m/s}$$

15.3 Step 2: Wind

Runway components ((11.3)–(11.4)), then the regulatory factor (12.2):

$$V_w = 10\cos(20^\circ - 20^\circ) = 10.0\ \text{kt} \qquad V_x = 10\sin(0^\circ) = 0.0\ \text{kt}$$
$$V_{w,\text{perf}} = 0.5 \times 10.0 = 5.0\ \text{kt} = 2.57\ \text{m/s}$$

The remaining calculations use the factored headwind of 5.0 kt.

15.4 Step 3: Weight and Takeoff Speeds

$$W = m\,g_0 = 18\,415.9 \times 9.80665 = 180\,598\ \text{N}$$
$$V_{S1} = \sqrt{\frac{2 \times 180\,598}{1.1094 \times 48.50 \times 1.57}} = 65.39\ \text{m/s} = 127.1\ \text{kt}$$
$$V_R = 1.10\,V_{S1} = 71.93\ \text{m/s} = 139.8\ \text{kt}$$
$$V_2 = 1.20\,V_{S1} = 78.47\ \text{m/s} = 152.5\ \text{kt}$$
$$V_{\text{TR}} = 1.15\,V_{S1} = 75.20\ \text{m/s} = 146.2\ \text{kt}$$

15.5 Step 4: Forces on the Ground Roll

Thrust ((6.8)–(6.10)). The ISA deviation of 17.6 °C exceeds the flat rating by 2.6 °C, so

$$\sigma_{\text{eff}} = \sigma(h,\ 2.6) = 0.9528$$
$$T(V) = 65\,149 \times 0.9528^{0.7}\left(1 - 0.90\,\frac{V}{349.0}\right)$$

Forces at brake release and at V_R ((7.4), $C_L = C_{L,\text{gnd}} = 0.30$, slope force $W \sin\theta$):

Quantity	**At $V_{\text{TAS}} = V_w$ (start)**	**At V_R**
Dynamic pressure q (Pa)	3.7	2 869.8
Lift L (N)	53	41 756
Drag D (N)	6	4 739
Thrust T (N)	62 561	51 298
Rolling friction $\mu_{\text{roll}}(W - L)$ (N)	3 611	2 777
Slope force $W \sin\theta$ (N)	298	298
Acceleration a (m/s^2)	3.185	2.361

Table 6. Challenger 3500: forces on the ground roll.

15.6 Step 5: Ground Roll by Numerical Integration

Equation (7.4) is integrated with RK45 from rest until $V_{\text{TAS}} = V_R$. The table gives selected samples from the numerical solution:

t **(s)**	V_{TAS} **(kt)**	a **(m/s^2)**	s **(ft)**
0.0	5.0	3.185	0
2.6	21.0	3.108	35
7.1	47.5	2.967	258
11.6	72.9	2.819	678
16.1	96.9	2.666	1 286
20.6	119.5	2.511	2 070
24.90	139.8 (V_R)	2.361	2 974

Table 7. Challenger 3500: ground-roll time history (RK45 output).

Rotation for t_{rot} (8.2):

$$s_{\text{rot}} = (71.93 - 2.57) \times 3 = 208.1\ \text{m}$$
$$s_{\text{ground}} = 906.5 + 208.1 = 1\,114.5\ \text{m} = 3\,657\ \text{ft}$$

15.7 Step 6: Air Distance to the Screen Height

Climb at V_2 with lift equal to weight ((7.8)–(7.9)); screen height 35 ft = 10.67 m:

$$q_{V_2} = \tfrac{1}{2} \times 1.1094 \times 78.47^2 = 3\,415.3\ \text{Pa}, \qquad C_{L,\text{climb}} = \frac{180\,598}{3\,415.3 \times 48.50} = 1.090$$

$$D = 3\,415.3 \times 48.50 \times (0.030 + 0.045 \times 1.090^2) = 13\,830\ \text{N}, \qquad T = 50\,237\ \text{N}$$

$$\sin\gamma = \frac{50\,237 - 13\,830}{180\,598} = 0.2016, \qquad \gamma = 0.2030\ \text{rad}, \qquad \text{gradient} = 20.6\,\%$$

Transition arc ((7.10)–(7.11)):

$$R_{\text{TR}} = \frac{75.20^2}{9.807 \times (1.2 - 1)} = 2\,883.1\ \text{m}$$

$$h_{\text{TR}} = 2\,883.1\,(1 - \cos 0.2030) = 59.19\ \text{m}$$

Since $h_{\text{TR}} \geq h_{\text{scr}}$, the screen is cleared inside the arc (7.12):

$$s_{\text{air}} = \sqrt{2\,883.1^2 - (2\,883.1 - 10.67)^2} = 247.8\ \text{m}$$

Corrected for wind (7.14), then the takeoff distance (8.4):

$$s_{\text{air}} = 247.8 \times \frac{75.20 - 2.57}{75.20} = 239.3\ \text{m}$$

$$\text{TOD}_{\text{AEO}} = 1\,114.5 + 239.3 = 1\,353.8\ \text{m} = 4\,442\ \text{ft}$$

15.8 Step 7: Balanced Field Length

For each trial V_1, the accelerate-go and accelerate-stop distances are computed ((9.1)–(9.2)) and their difference $f(V_1)$ (9.3):

V_1 (kt)	s_{go} (ft)	s_{stop} (ft)	$f(V_1)$ (ft)
69.9	7 586	1 298	6 288
87.4	7 094	2 038	5 056
104.9	6 438	2 957	3 481
118.8	5 772	3 828	1 945
129.3	5 176	4 564	612
139.8	4 482	5 375	−893
133.7	4 896	4 896	0.0

Table 8. Challenger 3500: accelerate-go and accelerate-stop against trial V_1.

f changes sign between the trial values, so Brent's method converges to the balanced $V_1 = 133.7$ kt

(bold row). Then ((9.4)–(9.5)):

$$\mathrm{BFL} = 4\,896\ \mathrm{ft}$$
$$1.15 \times \mathrm{TOD}_{\mathrm{AEO}} = 1.15 \times 4\,442 = 5\,108\ \mathrm{ft}$$
$$\mathrm{TOFL} = \max\left(4\,896,\ 5\,108\right) = 5\,108\ \mathrm{ft} = 1\,557\ \mathrm{m}$$

The TOFL is set by the all-engines distance with its 15% margin. The one-engine-inoperative climb gradient at V_2 is 6.3 %.

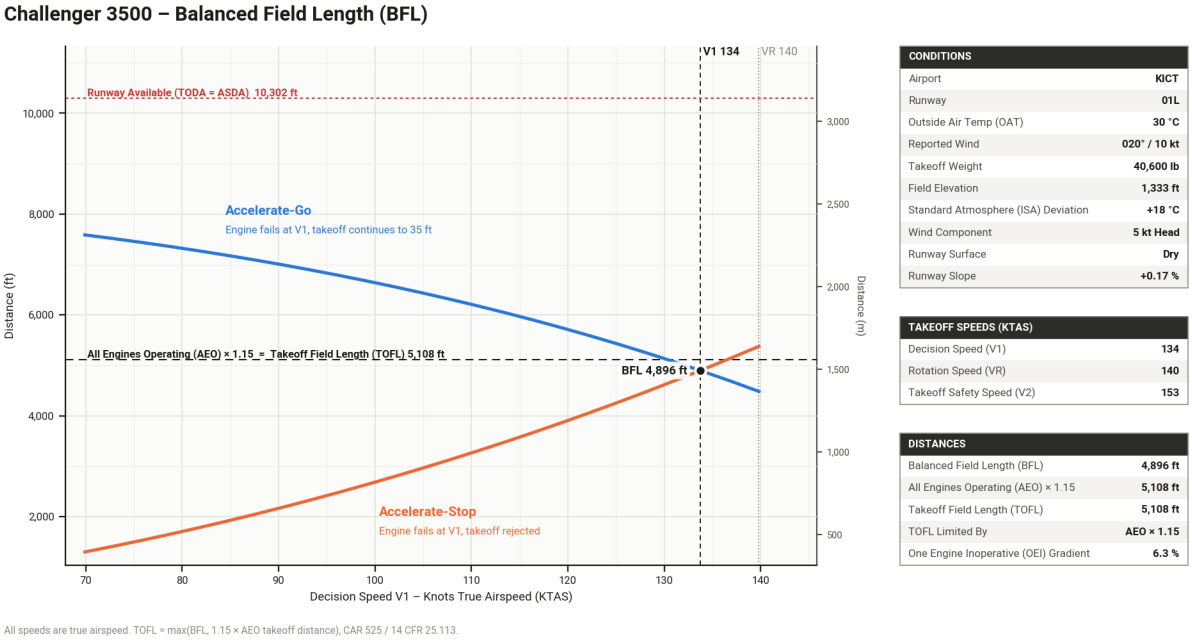


Figure 1. Challenger 3500: Balanced Field Length (BFL) for this example.

15.9 Step 8: Landing

Landing at MLW on the same runway, no reverse thrust credit ((10.1)–(10.5)):

$$V_{S0} = \sqrt{\frac{2 \times 151\,907}{1.1094 \times 48.50 \times 2.30}} = 49.55\ \text{m/s} = 96.3\ \text{kt}$$

$$V_{\text{REF}} = 1.30\,V_{S0} = 125.2\ \text{kt}, \qquad V_{\text{TD}} = 1.15\,V_{S0} = 110.8\ \text{kt}$$

$$s_{\text{air}} = \frac{15.24}{\tan 3^\circ} \times \frac{64.41 - 2.57}{64.41} = 279.2\ \text{m}$$

$$s_{\text{free}} = (56.98 - 2.57) \times 3 = 163.2\ \text{m}$$

$$s_{\text{brake}} = 367.7\ \text{m} \quad \text{((7.5) integrated to rest, 13.6 s)}$$

$$\text{LD} = 279.2 + 163.2 + 367.7 = 810.1\ \text{m} = 2\,658\ \text{ft}$$

$$\text{LDR} = \frac{810.1}{0.6} = 1\,350.2\ \text{m} = 4\,430\ \text{ft}$$

15.10 Step 9: Runway Check

Required	ft	Available	ft	Margin (ft)	Result
Takeoff Field Length (TOFL)	5 108	TODA	10 302	5 194	OK
Accelerate-Stop Distance (ASD)	4 896	ASDA	10 302	5 406	OK
Landing Distance Required (LDR)	4 430	LDA	10 302	5 872	OK

Table 9. Challenger 3500: runway check, KICT runway 01L.

16 Example B: Cirrus SR22T

16.1 Inputs

Table 10 summarises the aircraft inputs and operating conditions. The calculation is for Wichita Dwight D. Eisenhower National (KICT) runway 01L.

Input	Value
Maximum takeoff weight (MTOW)	3 600 lb = 1 632.9 kg
Maximum landing weight (MLW)	3 600 lb = 1 632.9 kg
Reference wing area, S	13.46 m^2 (144.9 ft^2)
C_{D_0} / k	0.027 / 0.045
$C_{L,\max}$ takeoff / landing	1.60 / 2.04
$C_{L,\text{gnd}}$	0.40
μ_{roll} / μ_{brake}	0.04 / 0.40
Sea-level static thrust, T_{SLS}	3 580 N (805 lbf)
Number of engines, N_e	1
Maximum power, $P_{\max}$	234.9 kW (315 hp)
Propeller efficiency, η_p	0.72
Turbocharger critical altitude	25 000 ft
Field elevation, h	1 333 ft = 406.3 m
Outside air temperature (OAT)	30 °C
Reported wind	20° / 10 kt
Runway true heading	20°
Runway slope (direction of travel)	0.165 %
Runway condition	Dry
TODA = ASDA / LDA	10 302 ft / 10 302 ft

Table 10. Cirrus SR22T: example inputs.

16.2 Step 1: Atmosphere

Standard temperature at the field elevation, then the ISA deviation from the OAT ((5.1) and (12.1)):

$$\Theta_{\text{std}} = 288.15 - 0.0065 \times 406.3 = 285.51\ \text{K}$$
$$\Delta T = 30.0 - (285.51 - 273.15) = 17.64\ ^\circ\text{C} \quad (\text{ISA } + 17.6)$$

Pressure, temperature, density, density ratio and speed of sound ((5.2)–(5.6)):

$$p = 101\,325\left(\frac{285.51}{288.15}\right)^{5.2559} = 96\,538\ \text{Pa}$$
$$\Theta = 285.51 + 17.64 = 303.15\ \text{K}$$
$$\rho = \frac{96\,538}{287.05 \times 303.15} = 1.1094\ \text{kg/m}^3$$
$$\sigma = \frac{1.1094}{1.225} = 0.9056$$
$$a = \sqrt{1.4 \times 287.05 \times 303.15} = 349.0\ \text{m/s}$$

16.3 Step 2: Wind

Runway components ((11.3)–(11.4)), then the regulatory factor (12.2):

$$V_w = 10\cos(20^\circ - 20^\circ) = 10.0\ \text{kt} \qquad V_x = 10\sin(0^\circ) = 0.0\ \text{kt}$$
$$V_{w,\text{perf}} = 0.5 \times 10.0 = 5.0\ \text{kt} = 2.57\ \text{m/s}$$

The remaining calculations use the factored headwind of 5.0 kt.

16.4 Step 3: Weight and Takeoff Speeds

$$W = m\,g_0 = 1\,632.9 \times 9.80665 = 16\,014\ \text{N}$$
$$V_{S1} = \sqrt{\frac{2 \times 16\,014}{1.1094 \times 13.46 \times 1.60}} = 36.61\ \text{m/s} = 71.2\ \text{kt}$$
$$V_R = 1.10\,V_{S1} = 40.27\ \text{m/s} = 78.3\ \text{kt}$$
$$V_2 = 1.20\,V_{S1} = 43.93\ \text{m/s} = 85.4\ \text{kt}$$
$$V_{\text{TR}} = 1.15\,V_{S1} = 42.10\ \text{m/s} = 81.8\ \text{kt}$$

16.5 Step 4: Forces on the Ground Roll

Thrust (6.7), turbocharged below its critical altitude so the power is not reduced ($\sigma_P = 1$):

$$T(V) = \min\left(1.0000 \times 3\,580,\ \frac{0.72 \times 1.0000 \times 234\,895}{V}\right)$$

Forces at brake release and at V_R ((7.4), $C_L = C_{L,\text{gnd}} = 0.40$, slope force $W \sin\theta$):

Quantity	**At $V_{\text{TAS}} = V_w$ (start)**	**At V_R**
Dynamic pressure q (Pa)	3.7	899.6
Lift L (N)	20	4 844
Drag D (N)	2	414
Thrust T (N)	3 580	3 580
Rolling friction $\mu_{\text{roll}}(W - L)$ (N)	640	447
Slope force $W \sin\theta$ (N)	26	26
Acceleration a (m/s^2)	1.783	1.649

Table 11. Cirrus SR22T: forces on the ground roll.

16.6 Step 5: Ground Roll by Numerical Integration

Equation (7.4) is integrated with RK45 from rest until $V_{\text{TAS}} = V_R$. The table gives selected samples from the numerical solution:

t **(s)**	V_{TAS} **(kt)**	a **(m/s^2)**	s **(ft)**
0.0	5.0	1.783	0
2.1	12.3	1.781	13
6.1	26.1	1.769	109
10.1	39.8	1.749	298
14.1	53.3	1.721	579
18.1	66.5	1.686	949
21.73	78.3 (V_R)	1.649	1 361

Table 12. Cirrus SR22T: ground-roll time history (RK45 output).

Rotation for t_{rot} (8.2):

$$s_{\text{rot}} = (40.27 - 2.57) \times 1 = 37.7\,\text{m}$$

$$s_{\text{ground}} = 414.9 + 37.7 = 452.6\,\text{m} = 1\,485\,\text{ft}$$

16.7 Step 6: Air Distance to the Screen Height

Climb at V_2 with lift equal to weight ((7.8)–(7.9)); screen height 50 ft = 15.24 m:

$$q_{V_2} = \tfrac{1}{2} \times 1.1094 \times 43.93^2 = 1\,070.6\,\text{Pa}, \qquad C_{L,\text{climb}} = \frac{16\,014}{1\,070.6 \times 13.46} = 1.111$$

$$D = 1\,070.6 \times 13.46 \times (0.027 + 0.045 \times 1.111^2) = 1\,190\,\text{N}, \qquad T = 3\,580\,\text{N}$$

$$\sin\gamma = \frac{3\,580 - 1\,190}{16\,014} = 0.1493, \qquad \gamma = 0.1498\,\text{rad}, \qquad \text{gradient} = 15.1\,\%$$

Transition arc ((7.10)–(7.11)):

$$R_{\text{TR}} = \frac{42.10^2}{9.807 \times (1.2 - 1)} = 903.8\,\text{m}$$

$$h_{\text{TR}} = 903.8\,(1 - \cos 0.1498) = 10.12\,\text{m}$$

Since $h_{\text{TR}} < h_{\text{scr}}$, the arc is followed by a steady climb (7.13):

$$s_{\text{air}} = 903.8 \sin 0.1498 + \frac{15.24 - 10.12}{\tan 0.1498} = 134.9 + 33.9 = 168.8\,\text{m}$$

Corrected for wind (7.14), then the takeoff distance (8.4):

$$s_{\text{air}} = 168.8 \times \frac{42.10 - 2.57}{42.10} = 158.5\,\text{m}$$

$$\text{TOD}_{\text{AEO}} = 452.6 + 158.5 = 611.1\,\text{m} = 2\,005\,\text{ft}$$

16.8 Step 7: Engine-Failure Case

With only one engine, the SR22T has no powered accelerate-go case after an engine failure. The model therefore uses AEO TOD for the takeoff requirement and does not calculate BFL.

16.9 Step 8: Landing

Landing at MLW on the same runway, no reverse thrust credit ((10.1)–(10.5)):

$$V_{S0} = \sqrt{\frac{2 \times 16\,014}{1.1094 \times 13.46 \times 2.04}} = 32.42\,\text{m/s} = 63.0\,\text{kt}$$

$$V_{\text{REF}} = 1.30\,V_{S0} = 81.9\,\text{kt}, \qquad V_{\text{TD}} = 1.15\,V_{S0} = 72.5\,\text{kt}$$

$$s_{\text{air}} = \frac{15.24}{\tan 3^\circ} \times \frac{42.15 - 2.57}{42.15} = 273.1\,\text{m}$$

$$s_{\text{free}} = (37.29 - 2.57) \times 2 = 69.4\,\text{m}$$

$$s_{\text{brake}} = 171.9\,\text{m} \quad \text{((7.5) integrated to rest, 9.5 s)}$$

$$\text{LD} = 273.1 + 69.4 + 171.9 = 514.4\,\text{m} = 1\,688\,\text{ft}$$

16.10 Step 9: Runway Check

Required	ft	Available	ft	Margin (ft)	Result
Takeoff Distance (TOD)	2 005	TODA	10 302	8 297	OK
Takeoff ground roll	1 485	TORA	10 302	8 817	OK
Landing Distance (LD)	1 688	LDA	10 302	8 614	OK

Table 13. Cirrus SR22T: runway check, KICT runway 01L.

17 Assumptions and Limitations

- Engine failure at $V_{\text{EF}} = V_1$, with no recognition delay before V_1; VMCG not checked.
- Fixed V-speed ratios ($V_R = 1.1V_{S1}$, $V_2 = 1.2V_{S1}$); no separate minimum-unstick speed (VMU) or liftoff speed (VLOF) schedule.
- Wet runway modelled only as halved braking friction; the Part 25 wet-runway takeoff rules are not modelled.
- OEI gradient reported gear-down at V_2; second-segment climb and weight, altitude and temperature (WAT) limits not computed.
- Idle thrust taken as zero when braking; brake energy limits ignored.
- Field elevation treated as pressure altitude; turbocharged power taken as independent of temperature below the critical altitude.
- Declared distances approximated from runway length and displaced thresholds.
- Aircraft aerodynamic coefficients are estimates. For each aircraft one value is calibrated to published data: $C_{L,\max}$ with takeoff flaps for the Challenger 3500 (to its 4 835 ft takeoff field

length [18]) and the static thrust for the SR22T (to its 1 517 ft ground roll [19]).

References

[1] D. P. Raymer, *Aircraft Design: A Conceptual Approach*, 6th ed. Reston, VA, USA: AIAA, 2018.

[2] A. Thompson and B. N. Taylor, "Guide for the use of the International System of Units (SI)," NIST, Gaithersburg, MD, USA, Spec. Publ. 811, 2008.

[3] J. D. Anderson, *Introduction to Flight*, 8th ed. New York, NY, USA: McGraw-Hill, 2016.

[4] International Civil Aviation Organization, *Manual of the ICAO Standard Atmosphere*, Doc 7488/3, 3rd ed. Montréal, QC, Canada: ICAO, 1993.

[5] *Airworthiness Standards: Normal Category Airplanes*, 14 C.F.R. Part 23.

[6] *Airworthiness Standards: Transport Category Airplanes*, 14 C.F.R. Part 25.

[7] Transport Canada, *Canadian Aviation Regulations*, Airworthiness Manual Ch. 525, *Transport Category Aeroplanes*.

[8] J. D. Anderson, *Aircraft Performance and Design*. New York, NY, USA: McGraw-Hill, 1999.

[9] S. Gudmundsson, *General Aviation Aircraft Design: Applied Methods and Procedures*. Oxford, U.K.: Butterworth-Heinemann, 2014.

[10] A. Filippone, *Advanced Aircraft Flight Performance*. Cambridge, U.K.: Cambridge Univ. Press, 2012.

[11] Airbus, "Getting to grips with aircraft performance," Flight Operations Support & Line Assistance, Blagnac, France, 2002.

[12] S. K. Ojha, *Flight Performance of Aircraft*. Washington, DC, USA: AIAA, 1995.

[13] E. Torenbeek, *Synthesis of Subsonic Airplane Design*. Delft, The Netherlands: Delft Univ. Press, 1982.

[14] W. A. Mair and D. L. Birdsall, *Aircraft Performance*. Cambridge, U.K.: Cambridge Univ. Press, 1992.

[15] *Landing Limitations: Destination Airports*, 14 C.F.R. §121.195.

[16] International Civil Aviation Organization, *Annex 14 to the Convention on International Civil Aviation: Aerodromes*, vol. I. Montréal, QC, Canada: ICAO.

[17] OurAirports, "Runway data." [Online]. Available: https://ourairports.com. Accessed: Oct. 21, 2026.

[18] Bombardier, “Challenger 3500 factsheet,” Montréal, QC, Canada, 2024.

[19] Aircraft Owners and Pilots Association, “Aircraft guide: Bombardier Challenger 350; Cirrus SR22T.” [Online]. Available: https://www.aopa.org

A Model Constants

Constant	Value	Meaning	Source
V_R/V_{S1}	1.1	Rotation speed ratio	[1]
V_2/V_{S1}	1.2	Takeoff safety speed ratio	[1], [6]
V_{TR}/V_{S1}	1.15	Transition speed ratio	[1]
n	1.2	Load factor in the transition arc	[1]
t_{rot}	1 s / 3 s	Rotation time, prop / jet	[1]
h_{scr}, takeoff	50 ft / 35 ft	Screen height, Part 23 / Part 25	[5], [6]
h_{scr}, landing	50 ft	Screen height, all aircraft	[6]
t_{stop}	2 s	Allowance at V1 before stopping	[6] §25.109
AEO factor	1.15	$\text{TOFL} \geq 1.15\,\text{TOD}_{AEO}$	[6] §25.113
V_{REF}/V_{S0}	1.3	Landing reference speed ratio	[1]
V_{TD}/V_{S0}	1.15	Touchdown speed ratio	[1]
Glideslope	3°	Approach path angle	[11]
t_{free}	2 s / 3 s	Free roll, prop / jet	[1]
Reverser cutoff	60 kt	Reversers stowed below	[11]
LDR factor	1/0.6	Landing within 60% of runway	[15] §121.195
Wet LDR factor	1.15	Wet runway	[15] §121.195
Wind factors	0.5 / 1.5	Headwind / tailwind	[6] §25.105
Wet braking	$0.5\,\mu_{brake}$	Wet runway friction	simplified
Integration step	0.5 s	RK45 maximum step	–

Table 14. Constants used in the model.